

```
In [1]: #Importing the Required Libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

```
In [2]: # Load iris data and store in dataframe
from sklearn import datasets
iris = datasets.load_iris()
dataset = pd.DataFrame(data=iris.data, columns=iris.feature_names)
dataset['target' ] = iris.target #target Column
dataset
```

```
Out[2]:
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	2
146	6.3	2.5	5.0	1.9	2
147	6.5	3.0	5.2	2.0	2
148	6.2	3.4	5.4	2.3	2
149	5.9	3.0	5.1	1.8	2

150 rows × 5 columns

```
In [3]: #Dividing in Training and Testing Dataset
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 4].values
```

```
In [4]: #Test_Train Split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20)
```

```
In [5]: #Fitting the dataset into the Model
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.fit_transform(X_test)
```

```
In [6]: #Applying KNN Model
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(X_train, y_train)
```

Out[6]: KNeighborsClassifier()

In [7]: `y_pred = classifier.predict(X_test) #Predicting Y value`

In [8]: `# Confusion Matrix`
`from sklearn.metrics import classification_report, confusion_matrix`
`print(confusion_matrix(y_test, y_pred))`
`print(classification_report(y_test, y_pred))`

```
[[ 8  0  0]
 [ 0 10  0]
 [ 0  3  9]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	8
1	0.77	1.00	0.87	10
2	1.00	0.75	0.86	12
accuracy			0.90	30
macro avg	0.92	0.92	0.91	30
weighted avg	0.92	0.90	0.90	30

In [9]: `error = []`

`# Calculating error for K values between 1 and 40`
`for i in range(1, 40):`
 `knn = KNeighborsClassifier(n_neighbors=i)`
 `knn.fit(X_train, y_train)`
 `pred_i = knn.predict(X_test)`
 `error.append(np.mean(pred_i != y_test))`

In [10]: `# Plotting the Graph`
`plt.figure(figsize=(12, 6))`
`plt.plot(range(1, 40), error, color='red', linestyle='dashed', marker='o',`
`markerfacecolor='blue', markersize=10)`
`plt.title('Error Rate K Value')`
`plt.xlabel('K Value')`
`plt.ylabel('Mean Error')`

Out[10]: `Text(0, 0.5, 'Mean Error')`

